Programming Language Principles, Design, and Implementation

Lecture 10 - Abstract Data Types

Sean Moss

University of Birmingham

Where are we?

- **Part 1: Principles of programming**
  - Functional languages
  - Operational semantics
  - Type systems
  - **Abstract datatypes**
- Part 2: Compilers

Today

Abstract data types

- What is data abstraction?
- Why data abstraction?
- Haskell modules
- Related concepts: type classes, objects

Further reading:

- "Types and Programming Languages", Pierce, Chapter 24

- Assignment 1 released tomorrow.
  - On material up to and including Lecture 9.
- Soon: another week 5 exercise sheet covering abstraction.
- Next Monday: consolidation (with me)
- Next Thursday: consolidation (with Vincent, followed by office hour)
- Next Friday: guest lecture by Professor Dan Ghica

Abstraction

Modularity:

- a good approach to problem solving is to divide a problem into smaller independent problems
- the same applies to programming
- implement solutions as collections of **independent modules**

Abstraction:

- the developer of a module should not have to fully understand another module to use it
- a module should be allowed to change without affecting the other modules, except from the module interface
- **abstraction**: ignore implementation details
- **information hiding**/**encapsulation**: hide the complexity of programs
- provide an abstract view of a program
- only need to understand the abstract view to use the program

Abstraction

Several forms of abstraction:

- **Functional abstraction**
- **Data abstraction**

Functional abstraction:

- separate the purpose of a function from its implementation
- to use a function we only need to know
  - what its **inputs** are
  - what its **outputs** are
  - its **pre-conditions**
  - its **post-conditions**
- we do not need to know how it works internally
- this is the **interface** of the function/module
- the implementation can change but not the interface

Functional Abstraction

Example:

```
1 sort :: Ord a => [a] -> [a]
```

- This is the type signature of Haskell's sorting function.
- It specifies its inputs/outputs.
- Its name indicates what it does.
- It requires the type a to be totally ordered.

This type signature is provided separately from the implementation.

Data Abstraction

Similarly, **data abstraction** abstracts over data:

- an **abstract data type (ADT)** does not specify how the data is implemented
- or how the functions manipulating the data are implemented

An ADT can be seen as

- an **abstract type name** A
- an **abstract interface** to these operations
- a **concrete representation type** T
- **concrete implementations** of operations on T

This forms an **abstraction barrier** between the implementation of a type and its uses.

- **inside** this "barrier", elements of this type can be seen as concrete elements of type T
- **outside** this "barrier", elements of this type are viewed abstractly as elements of type A

Data Abstraction

For example:

- We want to define an ADT for counters
- abstract name A: Counter
- concrete representation type T: **Integer**
- abstract interface:
  - new :: Counter
  - get :: Counter -> **Integer**
  - inc :: Counter -> Counter
- concrete implementation?
  - new = 0
  - get c = c
  - inc c = c + 1

Note:

- A Counter cannot be used as an **Integer** outside of the definition of this ADT.
- Should it be valid to write new+1 "outside" of the ADT? No
- The only operations on counters are get and inc

Data Abstraction

Another example:

- We want to define an ADT for collections of integers
- abstract name A: Collection
- concrete representation type T: [**Integer**]
- abstract interface:
  - add :: **Integer** −> Collection −> Collection
  - remove :: **Integer** −> Collection −> Collection
  - ...
- concrete implementation:
  - add i l =...
  - remove i l =...

Data Abstraction Using Modules

How can we define ADTs in Haskell?

Use modules: a module contains definitions of

- data types
- type declarations
- functions
- type signatures
- type classes
- instances of type classes
- and other kinds of declarations that we have not discussed yet

Note:

- a program is a collection of modules
- the top module is called "Main"
- by default only entities defined within a module are in scope
- we will see that we can also **import** other modules within a module

Data Abstraction Using Modules

Example—counter.hs:

```
1  module Counter where
2  type Counter = Integer
3
4  new :: Counter
5  new = 0
6
7  get :: Counter -> Integer
8  get c = c
9
10 inc :: Counter -> Counter
11 inc c = c + 1
```

Main module—main.hs (to interact with other modules):

```
1  module Main where
2  main = print "done"
```

Compile: ghc -o **mod** counter.hs main.hs

Data Abstraction Using Modules

Does this enforce the abstraction barrier we want?

The above example does not yet enforce an abstraction barrier:

```
1 module Main where
2 import Counter
3
4 break :: Counter -> Integer
5 break c = c
6
7 main = print "done"
```

We can access Counter's underlying representation, i.e., **Integer**

Data Abstraction Using Modules

1st step—we use a **datatype** instead:

```
1  module Counter where
2  data Counter = C Integer
3
4  new :: Counter
5  new = C 0
6
7  get :: Counter -> Integer
8  get (C c) = c
9
10 inc :: Counter -> Counter
11 inc (C c) = C (c + 1)
```

- This still does not enforce an abstraction barrier
- We have to hide the concrete implementation of Counter

Data Abstraction Using Modules

2nd step—we hide Counter's definition

```
1  module Counter(Counter,new,get,inc) where
2  data Counter = C Integer
3
4  new :: Counter
5  new = C 0
6
7  get :: Counter -> Integer
8  get (C c) = c
9
10 inc :: Counter -> Counter
11 inc (C c) = C (c + 1)
```

- the modules exports Counter, new, get, and inc
- it hides the type constructor C
- the only way to create/manipulate elements of type Counter is through new, get, and inc (its interface)

Data Abstraction Using Modules

We enforced an abstraction barrier

We now have to use get to implement our **break** function in Main

We can change the definition of Counter without affecting users of this ADT (as long as the interface does not change).

Constructors must be left out

If we were to export instead

```
1 module Counter(Counter(C),new,get,inc) where
```

then the code would not enforce an abstraction barrier anymore

We could implement **break** without get

Data Abstraction Using Modules

How would you define an ADT of queues?

For example:

```
1  module Queue(Queue,empty,enqueue,dequeue,peek) where
2  data Queue a = Q [a]
3
4  empty :: Queue a
5  empty = Q []
6
7  enqueue :: a -> Queue a -> Queue a
8  enqueue x (Q l) = Q (l ++ [x])
9
10 dequeue :: Queue a -> Queue a
11 dequeue (Q []) = Q []
12 dequeue (Q (_:l)) = Q l
13
14 peek :: Queue a -> Maybe a
15 peek (Q []) = Nothing
16 peek (Q (x:_)) = Just x
```

Related Concepts – Type Classes

Type classes provide a form of polymorphism called **ad-hoc polymorphism**.

They also provide a form of abstraction:

- A type class specifies an interface
- it specifies a class of types abstractly
- a member of that class much implement this interface

However, they do not enforce an abstraction barrier

For example, we saw the **Eq** type class in lecture 2:

```
1 class Eq a where
2   (==) :: a -> a -> Bool
```

Counter implements this abstraction as follows:

```
1 instance Eq Counter where
2   (C x) == (C y) = x == y
```

Related Concepts – Objects

Another way of abstracting over data is to use an **object-style**

An object includes:

- some internal state
- methods to manipulate the state

Similar to ADTs, the details of the internal state can be hidden

As opposed to ADTs where users only manipulate elements of the abstract data type, an object user manipulates a whole "package" of a state and methods

Conclusion

What did we cover today?

- What is data abstraction?
- Why data abstraction?
- Haskell modules
- Related concepts: type classes, objects

Next time?

- Existential types